



dsi | Departamento de
Sistemas e Informática

Generador de formas de ondas arbitrarias (AWG) con RPI Pico

Facultad de Ciencias Exactas, Ingeniería y Agrimensura
(UNR-FCEIA)

10 / 25

Índice

- 1 Introducción
- 2 Firmware
 - Programación Orientada a Objetos
 - Máquina de estados
 - Sistema Operativo en tiempo real
- 3 Hardware
- 4 Ensayos y video demostrativo
- 5 Repositorio

Introducción

- *¿Qué es el SASE?*

Introducción

- *¿Qué es el SASE?*
- *¿Cuáles fueron las motivaciones?*

Introducción

- *¿Qué es el SASE?*
- *¿Cuáles fueron las motivaciones?*
- *¿Qué es un AWG? ¿Porqué la elección de ese micro en particular?*

Simposio Argentino de Sistemas Embebidos (SASE) 2025

El Simposio Argentino de Sistemas Embebidos (SASE) es un evento anual que reúne a la comunidad académica y a la industria en torno a la temática de los Sistemas Embebidos.

Los objetivos del SASE son los siguientes:

- 1 Difundir en el ámbito profesional y académico las tecnologías asociadas a los sistemas embebidos.
- 2 Fomentar la interacción industria-academia en temas asociados a los sistemas embebidos.
- 3 Incentivar entre los estudiantes universitarios y los jóvenes profesionales el interés por los sistemas embebidos.



Universidad
Nacional
de San Martín



¿Qué es un AWG?

Un generador de formas de onda arbitrarias (AWG) permite generar funciones periódicas cargando puntos en un buffer y reproduciéndolos mediante un DAC a la frecuencia solicitada. La resolución depende de la profundidad del buffer y del DAC, afectando la frecuencia máxima alcanzable. Mayor resolución implicará menor frecuencia máxima.

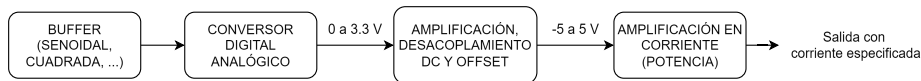


Figura 1.1: Diagrama en bloques del AWG

Raspberry Pi Pico

- **Microcontrolador:** RP2040, ARM Cortex-M0+ dual-core a 133 MHz.
- **Memoria:** 264 KB de SRAM y 2 MB de flash externa (QSPI).
- **Periféricos:** 2x I2C, 2x SPI, 2x UART, 16 canales PWM, ADC de 12 bits (3 canales + temperatura interna).
- **PIO:** 2 bloques con 4 máquinas de estado programables cada uno.
- **GPIO:** 26 pines (3.3 V, multifunción, no toleran 5 V).
- **USB:** 1.1, modo host o dispositivo.
- **Alimentación:** 1.8–5.5 V (regulador interno a 3.3 V).
- **Programación:** C/C++, MicroPython, Arduino.

Raspberry Pi Pico

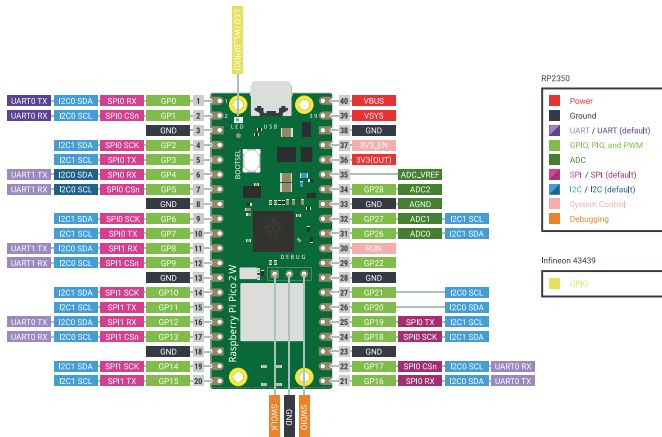


Figura 1.2: Pinout de la RPI Pico

Programmable Input/Output (PIO) y DMA

- **DMA (Direct Memory Access):** transfiere datos entre memoria y periféricos sin intervención del CPU, liberando carga de procesamiento. Cuenta con 12 canales independientes y se usa comúnmente con **PIO**, **SPI** o **ADC** para transferencias continuas de datos. *En nuestro caso lo utilizamos para transferir el buffer en la memoria RAM al PIO.*

Programmable Input/Output (PIO) y DMA

- **DMA (Direct Memory Access):** transfiere datos entre memoria y periféricos sin intervención del CPU, liberando carga de procesamiento. Cuenta con 12 canales independientes y se usa comúnmente con **PIO**, **SPI** o **ADC** para transferencias continuas de datos. *En nuestro caso lo utilizamos para transferir el buffer en la memoria RAM al PIO.*
- **PIO (PROGRAMMABLE I/O):** es un periférico programable que permite implementar protocolos personalizados o de alta velocidad directamente en hardware, sin cargar la CPU. Posee 2 bloques con 4 máquinas de estado cada uno. Es ideal para bit-banging preciso o interfaces como WS2812, I2S o DVI. En este caso fue utilizado para generar la señal de salida en la escalera **R-2R**.

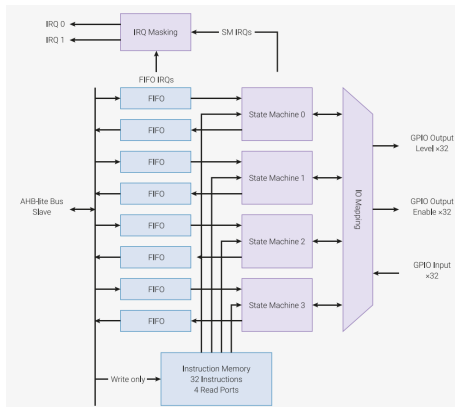


Figura 1.3: Diagrama de niveles en bloques del periférico PIO.

FIRMWARE

```
31 def __init__(self):
32     self.file = None
33     self.fingerprints = set()
34     self.logdupes = True
35     self.debug = debug
36     self.logger = logging.getLogger(__name__)
37     if path:
38         self.file = open(os.path.join(path, "requests.log"),
39                         "a")
40         self.file.seek(0)
41         self.fingerprints.update(ex.request() for e in self.requests)
42
43 @classmethod
44 def from_settings(cls, settings):
45     debug = settings.getbool("debug", False)
46     return cls(job_dir(settings), debug)
47
48 def request_seen(self, request):
49     fp = self.request_fingerprint(request)
50     if fp in self.fingerprints:
51         return True
52     self.fingerprints.add(fp)
53     if self.file:
54         self.file.write(fp + os.linesep)
55
56 def request_fingerprint(self, request):
57     return request_fingerprint(request)
```

Programación orientada a objetos (POO)

¿Cuál fue el enfoque de la programación?

Programación orientada a objetos (POO)

¿Cuál fue el enfoque de la programación?

El firmware fue escrito basado en el concepto de la *programación orientada a objetos*. Para esto mismo cada archivo/objeto representa una porción específica del hardware. En especial pueden destacarse las etapas de **amplificación, desacoplamiento DC y filtros PB** (offset).

Programación orientada a objetos (POO)

¿Cuál fue el enfoque de la programación?

El firmware fue escrito basado en el concepto de la *programación orientada a objetos*. Para esto mismo cada archivo/objeto representa una porción específica del hardware. En especial pueden destacarse las etapas de **amplificación, desacoplamiento DC y filtros PB** (offset).

Además se hizo énfasis en una organización en forma de **capas**, con la idea de que las capas asociadas a **niveles superiores** representen una mayor interacción con el usuario, mientras que capas de **niveles inferiores** una mayor abstracción para este.

Programación Orientada a Objetos

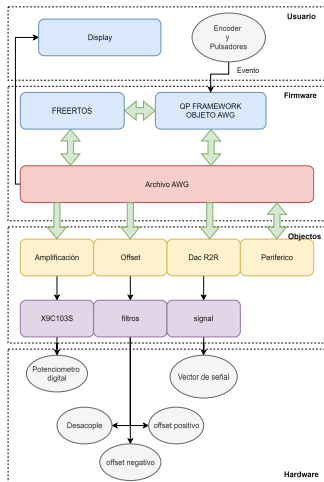


Figura 2.1: Estructura en capas del firmware.

Máquina de estados

Con la intención de generar un **menú de configuración** para los distintos parámetros de la señal como ***amplitud, frecuencia, offset*** y ***tipo de señal***, se diseñó una **máquina de estados**. Utilizamos el **framework de Quantum Leaps**, en el que desarrollamos de forma gráfica los distintos estados, eventos y temporizadores a utilizar. Podemos ver una representación de esta máquina de estados en la figura 2.2.



Máquina de estados

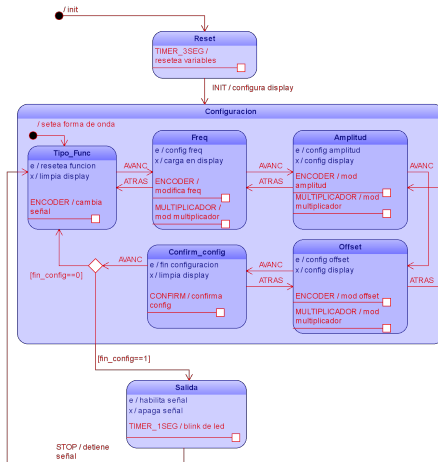


Figura 2.2: Máquina de estados.

Sistema Operativo en Tiempo Real

- *¿Cómo están distribuidas las tareas?*
- *¿Cómo se relaciona con la MEF de antes?*

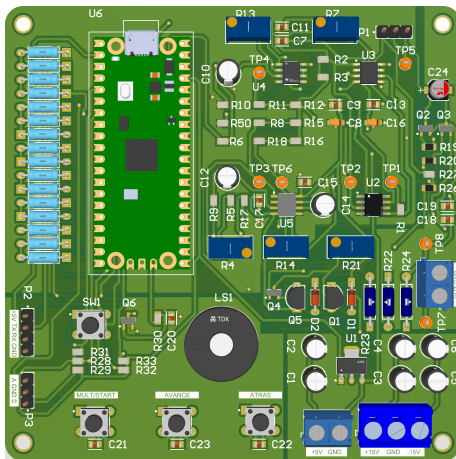
Sistema Operativo en Tiempo Real

- *¿Cómo están distribuidas las tareas?*
- *¿Cómo se relaciona con la MEF de antes?*

Para la gestión del firmware se utilizó FreeRTOS, elegido por ser de código abierto y capaz de integrarse con Quantum Leaps. La máquina de estado se ejecuta como una tarea estática, mientras que en paralelo corren otras tareas esenciales: detección de pulsadores, manejo de interrupciones del encoder y generación de la señal (AWG) que son dinámicas.



HARDWARE



Hardware

El hardware se compone de varias etapas que permiten producir una señal de acuerdo a las especificaciones del usuario. Se mencionan las siguientes:

- **Conversión analógica digital**

Hardware

El hardware se compone de varias etapas que permiten producir una señal de acuerdo a las especificaciones del usuario. Se mencionan las siguientes:

- **Conversión analógica digital**
- **Amplificación**

Hardware

El hardware se compone de varias etapas que permiten producir una señal de acuerdo a las especificaciones del usuario. Se mencionan las siguientes:

- **Conversión analógica digital**
- **Amplificación**
- **Desacople DC**

Hardware

El hardware se compone de varias etapas que permiten producir una señal de acuerdo a las especificaciones del usuario. Se mencionan las siguientes:

- **Conversión analógica digital**
- **Amplificación**
- **Desacople DC**
- **Offset de continua**

Hardware

El hardware se compone de varias etapas que permiten producir una señal de acuerdo a las especificaciones del usuario. Se mencionan las siguientes:

- **Conversión analógica digital**
- **Amplificación**
- **Desacople DC**
- **Offset de continua**

¿Qué hace a grandes rasgos cada etapa? ¿En qué radica su importancia?

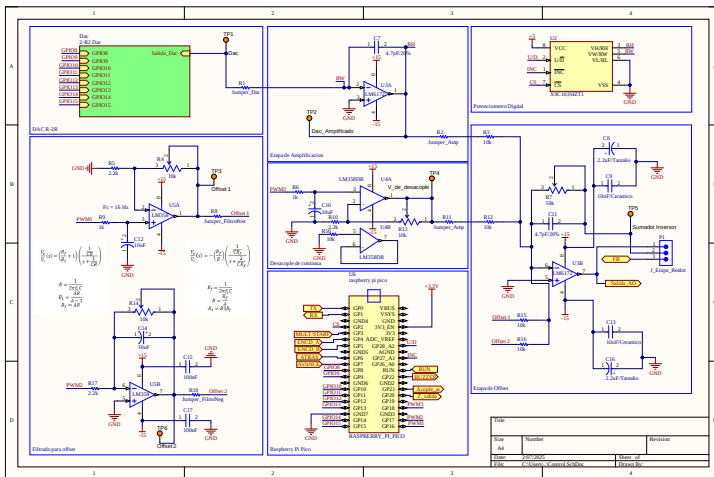
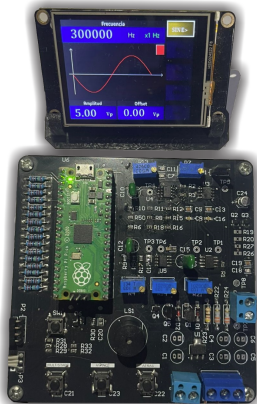


Figura 3.1: Etapa de Control.

ENSAYOS



Ensayos realizados

Senoidal:

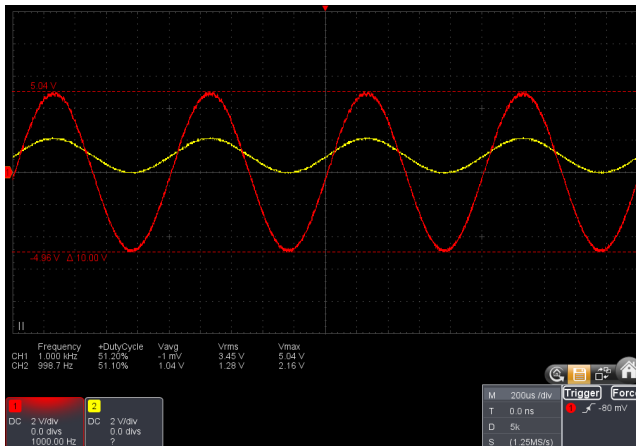


Figura 4.1: 10 V_{pp} y $f = 10$ KHz.

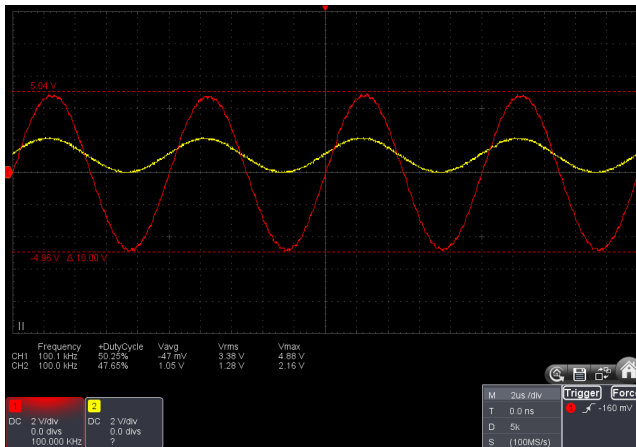


Figura 4.2: 10 V_{pp} y $f = 100$ KHz.

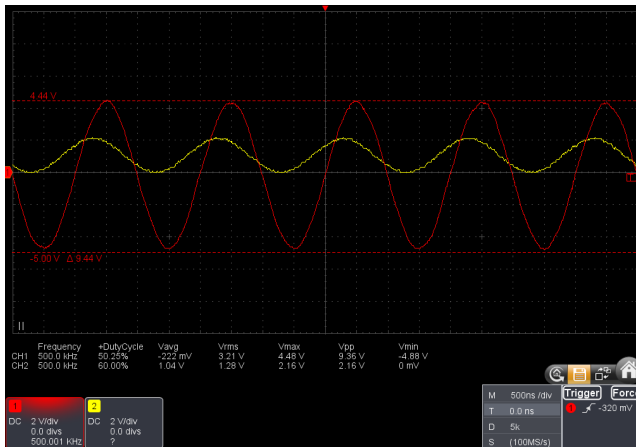


Figura 4.3: $10 V_{pp}$ y $f = 500$ KHz.

Cuadrada:

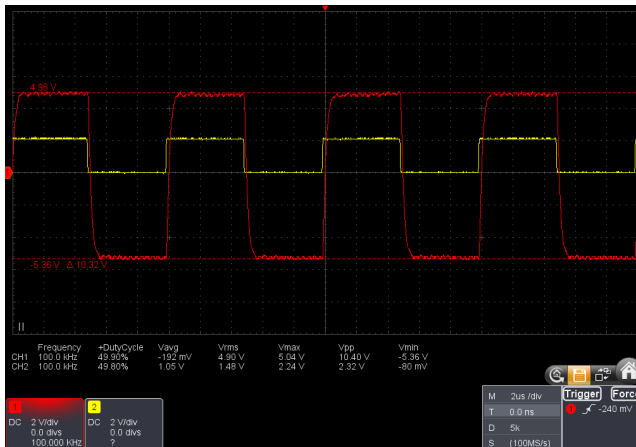


Figura 4.4: 10 V_{pp} y $f = 100$ KHz.

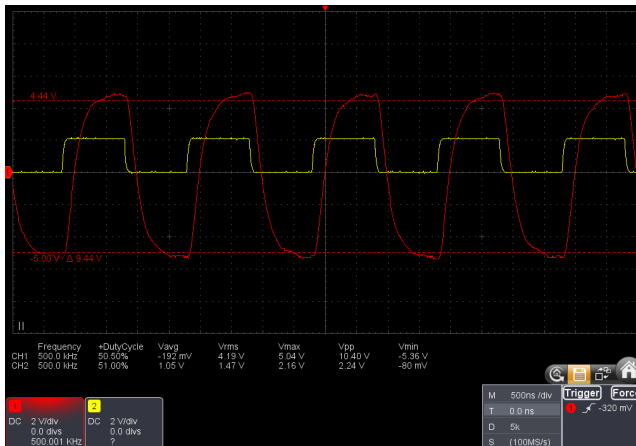
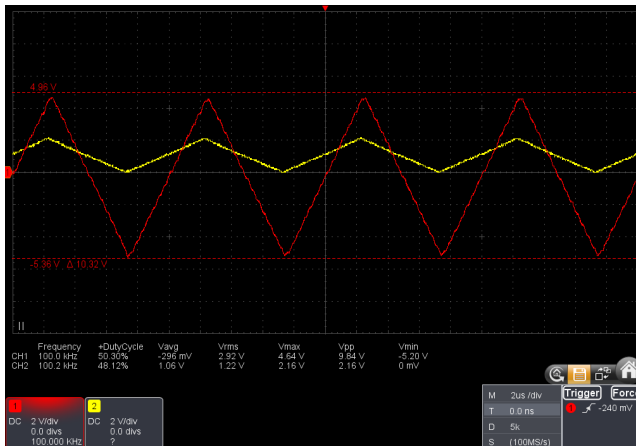
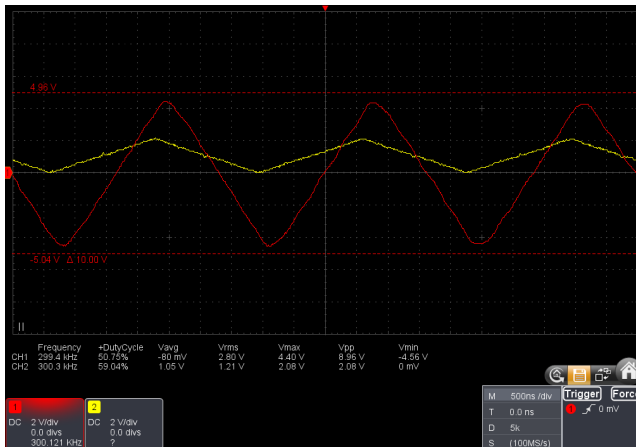


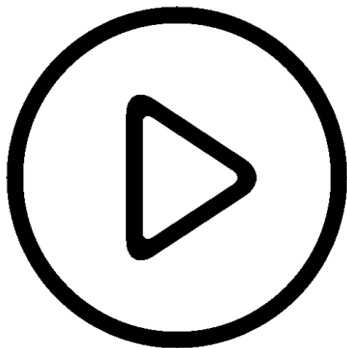
Figura 4.5: 10 V_{pp} y $f = 500$ KHz.

Triangular:





Video demostrativo



Repositorio

Emails:

- Agustin Zuliani:
amzuliani02@gmail.com
- Daniel Márquez:
dmarquez@fceia.unr.edu.ar



Gracias por mirar y escuchar!